SYNTHESIS REPORT

Isolation Approaches for Concurrent AI Coding Agents

A synthesis of architectures, implementation mechanics, and documented trade-offs across workspace, runtime, and coordination isolation strategies

Synthesized from three independent research documents

Compiled July 17, 2026

Contents

Abstract

Running multiple AI coding agents against the same codebase concurrently raises a distinct systems problem: agents must be prevented from corrupting each other's files, processes, services, credentials, and final merged result. This paper synthesizes three independent research documents on the topic into a single descriptive account of the isolation approaches in current use. The sources agree that "isolation" is not a single mechanism but a stack of largely orthogonal layers — context, source tree, process/runtime, external state, credentials/network, and integration — and that products differ mainly in which layers they address and with which primitive. We present a unified taxonomy of nine mechanism families (shared checkout with permission gating, branch-only separation, Git worktrees, full clones, containers with snapshotting, copy-on-write filesystem branching, sandboxed runtimes and microVMs, OS-level process sandboxes, and logical orchestration-layer isolation), describe each mechanism's architecture and implementation mechanics as documented in the sources, and tabulate reported trade-offs. We further cover cross-cutting implementation challenges (dependency drift, untracked-file propagation, long-running services, shared daemons, secrets handling, per-workspace indexing), the integration and coordination layer that isolation does not replace (semantic conflicts, merge queues, leases, ownership partitions), the product landscape across direct-API and subscription access models, and the usage patterns the sources document for recurring scenarios. The treatment is descriptive throughout: the aim is to record how each approach works and what trade-offs have been reported, not to advocate a particular design.

1 Introduction

AI coding agents have moved from single-session assistants to systems in which several agents work concurrently, either under a human operator's direction or as unattended fleets. Running N agents against one repository R at the same time creates collision risks that a single agent never faces: concurrent edits to the same files, concurrent mutation of shared dependency directories and databases, contention for ports and credentials, and the problem of combining many parallel change streams back into one coherent codebase.[S2] A growing ecosystem of applications and orchestration frameworks — spanning both direct-API (bring-your-own-key) and subscription-based access models — has emerged to manage this concurrency, and each makes different choices about how agents are isolated from one another.[S3]

This paper synthesizes three independent research documents on that design space into a single account. The three sources were produced separately, with different scopes and emphases, and partially overlapping coverage; the synthesis merges their taxonomies, deduplicates their claims, preserves their implementation-level detail, and notes points where their classifications differ. The intent is descriptive rather than prescriptive: where the sources characterize an approach's strengths, weaknesses, or typical usage, that characterization is reported and attributed, not endorsed. Table 1 identifies the source documents.

Table 1. Source documents synthesized in this paper

ID	Document	Date	Character

[S1]	*Multi-Agent Coding Isolation: Architectures, implementations, and trade-offs for concurrent AI software agents*	July 16, 2026	Layered-stack analysis; product landscape; reference architecture; vendor evaluation criteria
[S2]	Untitled research compilation on multi-agent isolation strategies (`isolation-research.md`)	2026	Mechanism deep-dives with implementation mechanics; six-category taxonomy; scenario mappings; convergence analysis
[S3]	*Isolation Approaches for Parallel AI Coding Agents — A Deep Research Report*	July 17, 2026	Mechanism-by-mechanism survey of the 2025–2026 tool landscape; sandbox infrastructure data; merge-layer tooling; access-model analysis

The three sources approach the subject from complementary angles. [S1] analyzes isolation as a six-layer stack and emphasizes that no single layer substitutes for the others; it includes a product-landscape comparison, a reference architecture, and a set of vendor-evaluation questions. [S2] is the most implementation-oriented: it specifies bootstrap commands, filesystem layouts, snapshotting strategies, and a taxonomy organized around mechanism internals, plus a set of "gotchas" drawn from operational experience. [S3] is the broadest survey of named tools and infrastructure providers, with reported startup-latency figures, merge-coordination tooling, and an analysis of the subscription-versus-API access dimension. Where the sources classify the same mechanism differently — for example, the isolation strength attributed to Git worktrees — the discrepancy is resolved by decomposing the mechanism by layer, as described in Section 2.

2 Problem Framing

2.1 Collision domains

[S2] frames the problem formally: when *N* agents operate on a single repository concurrently, three distinct collision domains arise.[S2] **State collision** occurs when one agent reads a file, another edits it, and the first then writes or reasons from stale content; without separation this extends to concurrent `git add` operations corrupting the shared index, and to agents building mental models of a working tree that no longer matches disk.[S3] **Execution collision** occurs when agents' commands interfere — one agent's test run racing another's dependency installation, port bindings conflicting on localhost, or package caches being mutated mid-build. **Identity and permission collision** covers the need for distinct git authorship, API keys, and sandbox credentials per agent. A fourth domain appears across all three sources under different names — the **integration** domain: even perfectly isolated work streams must eventually be merged, and merge is where semantic incompatibilities surface (Section 6).

2.2 Isolation as a layered stack

[S1]'s central framing is that a multi-agent coding system is a stack of isolation layers that may be provided by one product or assembled from separate tools, and that conflating the layers produces misleading claims — "parallel agents are isolated" may mean only that their chat histories are separate.[S1] Table 2 reproduces the six layers with the failure mode each addresses. Three non-equivalences recur throughout the literature and are worth stating explicitly:

Table 2. The isolation stack: layers, scope, and failure modes addressed (after [S1])

Layer	What is isolated	Failure mode addressed
A. Context	Prompts, transcripts, tool results, temporary memory, token budget	Reasoning contamination; accidental cross-task context
B. Source tree	Working files, Git index, branch, untracked files, generated outputs	Edit collisions, branch switching, resets, mixed diffs
C. Process / OS	Processes, dependencies, package installation, CPU, memory, mounts	Port conflicts, process interference, host filesystem damage, resource exhaustion
D. External state	Databases, caches, queues, object stores, browser profiles, volumes	Cross-task data corruption; nondeterministic tests
E. Credentials / network	Secrets, repository tokens, cloud permissions, outbound connectivity	Credential leakage, excessive privilege, supply-chain exposure
F. Integration	Rebase, test, review, merge ordering, deployment policy	Textual and semantic conflicts reaching the target branch

> **Context isolation is not workspace isolation.**
> Subagents may hold independent conversations while sharing one working directory; a separate context window aids focus and token management but is not a concurrency boundary.[S1]

> **Source-tree isolation is not security isolation.**
> A Git worktree gives each task an independent editable tree, but all agents may still run as the same OS user, colliding through processes, ports, global caches, environment variables, the Docker daemon, or files outside the repository.[S1]

> **Runtime isolation is not integration safety.**
> Two tasks can run in perfectly isolated VMs and still produce mutually incompatible changes; runtime boundaries prevent interference during execution, not conflicts in architecture, API, schema, or merge order.[S1]

These distinctions reconcile the sources' different strength ratings. [S2] rates worktree isolation "High (Git), Medium (FS)"; [S1] rates worktrees high for edit isolation but none for runtime containment; [S3] describes worktrees as file-level isolation only. Decomposed by layer, all three agree: worktrees address layer B thoroughly and layers C–F not at all. The same decomposition applies to full clones, which [S2] groups with virtualized environments under "workspace forking" at "High (FS + Git + Process)" strength, while [S1] notes a clone by itself provides no process or security boundary — both statements are accurate once the environment the clone runs *in* is specified.

3 A Unified Taxonomy of Isolation Approaches

Each source proposes its own taxonomy: [S2] defines six categories organized by mechanism internals; [S1] enumerates eight workspace/runtime strategies ordered by containment strength; [S3] surveys the tool

landscape along a spectrum from convention-only sharing to per-task cloud machines. Merging the three yields the nine mechanism families in Table 3. The families are not mutually exclusive — deployed systems typically stack several (for example, a worktree inside a container, coordinated by an orchestration layer), a point all three sources emphasize.[S1][S3]

Table 3. Unified taxonomy of isolation mechanism families, merged from [S1], [S2], and [S3]

Mechanism family	What it isolates	What it does not isolate	Documented examples
Shared checkout with permission gating or optimistic patching	Nothing structural; approval gates or patch-apply discipline	Files, Git state, runtime, secrets	IDE-native diff-apply flows; default single-checkout agent sessions; task-claiming protocols[S2][S3]
Branch-only separation in one checkout	History namespace	Simultaneous filesystem state (one index, one working tree)	Sequential pipelines[S1]
Git worktree per agent	Working tree, index, HEAD, branch	Runtime, ports, services, untracked files, secrets	Claude Code `--worktree`; Cursor `/worktree`; Codex worktree mode; Conductor; Superset; Claude Squad; Vibe Kanban; Gas Town; gnhf[S3]
Full clone per agent	Filesystem and independent Git metadata	Process, kernel, same-machine runtime	CI-style runners; ephemeral cloud workers; the "workspace forking" bootstrap pattern[S1][S2]
Container per agent, with image snapshotting	Filesystem, dependencies, env vars, PID/NET namespaces	Host kernel; Docker daemon (if socket-mounted); semantic conflicts	Sculptor; Dagger Container Use; OpenHands runtimes; Factory "persistent compute"; devcontainer-based agents; Daytona[S2][S3]
Copy-on-write filesystem branching	Filesystem deltas over a shared read-only base	Process and kernel; external side effects	OverlayFS views (Replit, Gitpod, StackBlitz); Factory snapshot branching; APFS `cp -c` clones; ZeroBoot VM-fork research[S2][S3]
Sandboxed container runtime (userspace kernel)	Syscall surface between workload and host kernel	Workspace separation (orthogonal concern)	gVisor (Modal, Claude.ai web sandbox, GKE Agent Sandbox); Kata Containers[S1][S3]
MicroVM / full VM per task	Guest kernel boundary; strongest common runtime isolation	Git and semantic conflicts	Firecracker-based E2B, Vercel Sandbox, Fly Sprites; per-task cloud VMs in Jules, Devin, Codex cloud, Copilot coding agent[S1][S3]

OS-level process sandbox	Process filesystem writes and network egress	Which files concurrent agents edit (orthogonal)	Anthropic `sandbox-runtime` (Seatbelt/bubblewrap); Codex CLI sandbox (Landlock/seccomp/restricted tokens)[S3]
Logical isolation at the orchestration layer	Run state, thread IDs, task claims, mailboxes	Requires an infrastructure primitive underneath	LangGraph checkpointers; Temporal; AutoGen; task-claim systems[S2]

Figure 1 positions the families on two axes documented in [S3]: containment strength and the typical time to reach a ready-to-code environment. The cloud end of the spectrum inverts the historical cost curve — pre-warmed snapshot pools bring microVM startup into the sub-second range, so the latency argument that once favored lightweight local mechanisms no longer decides the question by itself.[S3]

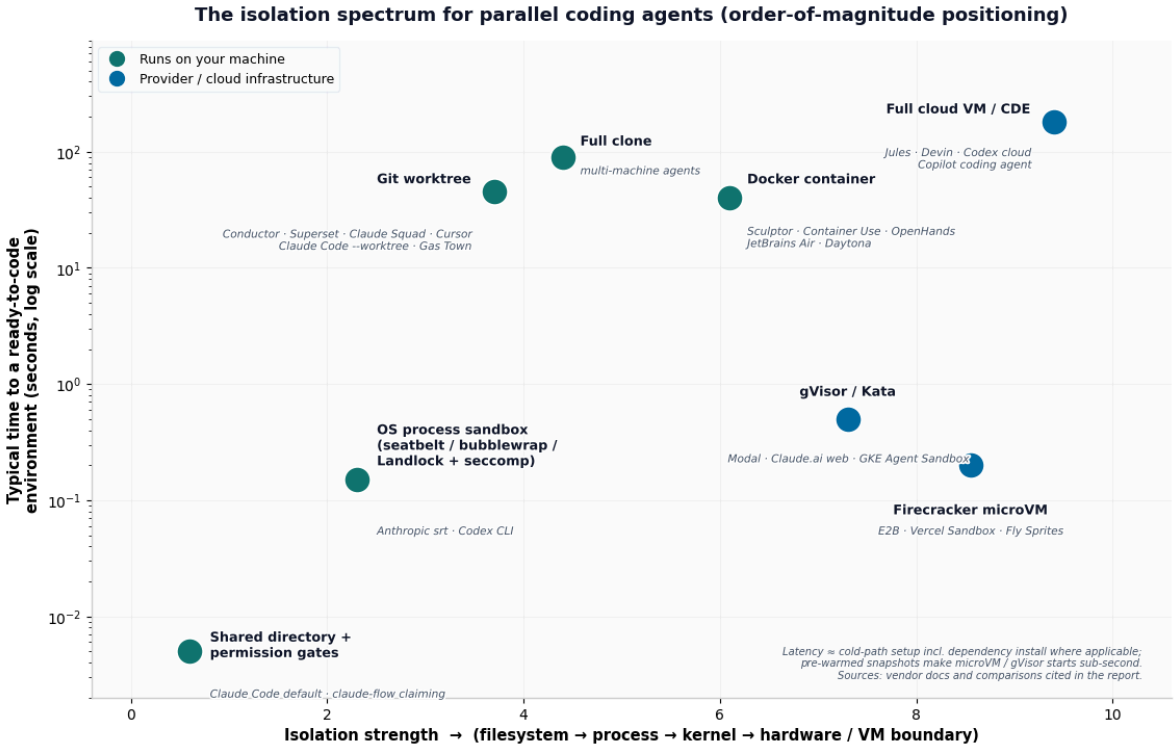

Figure 1. The isolation spectrum for parallel coding agents: containment strength versus typical time to a ready-to-code environment (order-of-magnitude positioning, from [S3]; latencies include dependency setup where applicable, with pre-warmed snapshots accounting for sub-second microVM and userspace-kernel starts).

4 Isolation Mechanisms: Architecture and Mechanics

This section walks each mechanism family, recording the architecture, the implementation mechanics given in the sources, documented deployments, and reported trade-offs. Figure 2 shows the task lifecycle that most environment-provisioning approaches share, regardless of which primitive materializes the workspace.

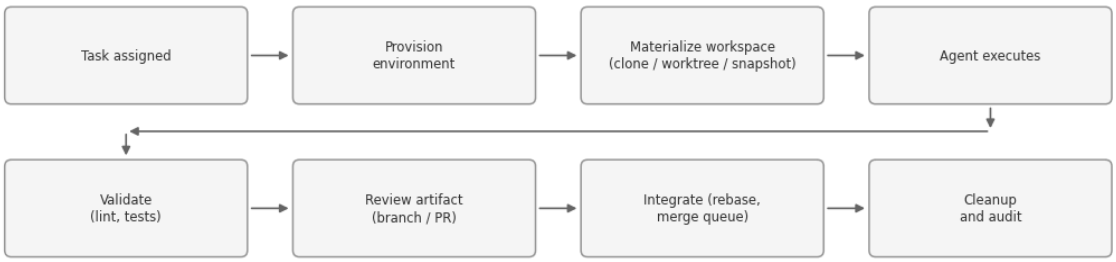

Figure 2. The shared task lifecycle across provisioning-based isolation approaches (synthesized from [S1] and [S2]). The workspace materialization step is where the mechanism families differ; the integration step is covered in Section 6.

4.1 Shared checkout and optimistic concurrency

Architecture. All agents operate in a single working directory; correctness rests on approval gates, patch discipline, or claiming protocols rather than on any structural boundary. [S2] describes the IDE-native variant: the agent proposes a diff rather than writing directly, the IDE applies the patch to an in-memory editor buffer, a human accepts explicitly or implicitly, and only then is the result written to disk and the Git index — "the IDE *is* the lock manager."[S2] The background-agent variant documented for Cursor spawns a headless editor instance with its own copy of the workspace (via directory copy or an internal worktree), communicates with the main IDE over IPC, and returns a diff for the user to merge.[S2]

Claiming protocols. A more structured variant replaces human gating with advisory task claims: an agent declares "I am working on this; do not touch it," and the claim is released when the agent finishes or dies. [S3] documents this pattern in claude-flow (shared codebase, in-memory claiming) and in Claude Code's Agent Teams, where teammates share a working directory by default and coordinate through a file-based task list with owner-checked updates; worktree isolation is available as an explicit opt-in.[S3] The claim is advisory rather than enforced — nothing at the filesystem level prevents a confused agent from editing a claimed file.

Documented trade-offs. Strengths: zero setup, zero startup latency, no merge step, immediate visibility of edits, shared caches.[S1] Limitations: uncommitted changes collide; tests observe mixed states; formatters and generators rewrite shared files; `reset`, `checkout`, `clean`, or `stash` can destroy other work; attribution becomes difficult; race conditions arise when a background agent and a foreground user edit the same file.[S1][S2] [S1] characterizes the configuration as acceptable for a sequential planner–reviewer–editor pipeline with a single writer, and unsuitable for independent concurrent tasks; [S2] characterizes the IDE-mediated variant as inherently human-in-the-loop and therefore unsuited to unattended fleets, additionally noting the RAM cost of multiple headless editor instances.

4.2 Git worktrees

Architecture. A Git worktree attaches an additional working directory to one repository: each worktree has its own checked-out branch, working files, `HEAD`, and index, while the object database, refs, remotes, hooks,

and configuration remain shared.[1] Internally, the new directory contains a `.git` *file* pointing to `.git/worktrees/<name>`, with Git splitting per-worktree state (`$GIT_DIR`) from shared state (`$GIT_COMMON_DIR`).[S3] [S2] gives the canonical layout:

```
/repo            (main worktree or bare)
/worktrees/
    agent-1/     -> .git file points to /repo/.git/worktrees/agent-1
    agent-2/
```

Documented implementations. [S3] catalogues first-party support: Claude Code's `claude --worktree <name>` branches from the default remote branch into `<repo>/.claude/worktrees/<name>`, auto-removes clean worktrees on exit, accepts `isolation: "worktree"` in subagent frontmatter, and exposes a `WorktreeCreate` hook for per-worktree setup commands; its background-agent lifecycle is branch → temp worktree → run → report path and branch → auto-clean if unchanged.[S3] Cursor 3's Agents Window exposes `/worktree`, `/apply-worktree`, and `/delete-worktree`, with a `/best-of-n` mode that races several models on one task in separate worktrees.[S3] OpenAI's Codex application offers local, cloud, and worktree modes with per-environment sandbox settings.[S3] The orchestrator class — Conductor, Superset, Claude Squad, Vibe Kanban, Nimbalyst, Parallel Code, Emdash, Gas Town, and gnhf's `--worktree` flag — is characterized in [S3] as a management layer over this single primitive, typically adding setup/teardown scripts, per-workspace environment files, and status dashboards.[S3]

Documented pitfalls. All three sources converge on the same list. (i) *Untracked files do not propagate*: a fresh worktree contains tracked files only — no `node_modules`, no virtualenvs, no `.env` — so per-workspace setup scripts, "files to copy" conventions (Conductor's `.worktreeinclude`), copy-on-write directory clones on APFS (`cp -c`), or shared dependency symlinks are used; [S3] notes symlinking `node_modules` corrupts the shared folder once two worktrees install different versions.[S3] (ii) *One branch, one worktree*: Git refuses to check out the same branch twice, so scaled systems use unique branch names per session or detached `HEAD` (the workaround documented for Codex).[S3] (iii) *Worktrees share the runtime*: ports, databases, environment variables, package caches, and test state remain common — [S3] quotes the operational heuristic that five agents each starting a dev server on port 3000 fail loudly, while five agents against one shared test account fail silently.[S3] [S2] records the standard mitigations: distinct ports (e.g., `PORT=$AGENT_ID+3000`), distinct database filenames, or a compose stack per worktree.[S2] (iv) *Worktrees defer conflicts rather than resolving them* — covered in Section 6.

Documented trade-offs. Strengths: near-instant creation and low storage overhead from the shared object database; isolated uncommitted edits; natural branch-per-task review flow in which each agent's output is an ordinary, reviewable branch; no infrastructure beyond Git itself.[S1][S3] Limitations: no process or security boundary — agents run as the same OS user with full filesystem reach; some tools mishandle linked `.git` layouts; generated build trees still consume per-worktree storage; cleanup (`git worktree prune`) is an operational burden.[S1] [S1] describes worktrees as the default source-isolation mechanism for many local coding-agent workflows; [S3] documents their adoption across essentially every local orchestrator surveyed.

4.3 Full repository clones

Architecture. Each task receives an independent `git clone`, including independent Git metadata — [S2]'s "workspace forking" pattern: provision a fresh environment, `git clone --depth=1 --branch <base> <url> /workspace` (shallow clones and sparse checkout for monorepos), let the agent operate, commit and push, open a pull request, and destroy the environment. Only the Git history and artifacts (logs, test results, screenshots) leave the sandbox; the working directory is ephemeral. Concurrency is trivial: *N* tasks = *N* clones, with zero shared state.[S2]

Documented trade-offs. Strengths: a simple mental model; strong filesystem independence; portability between machines and containers; broad tool compatibility; deterministic reproducibility ("run it again" is a fresh clone); containment of untrusted setup scripts when the clone runs in a disposable VM.[S1][S2] Limitations: repeated network transfer, disk use, and dependency installation; Git-host load; cold starts of roughly 30 seconds to 5 minutes per task when dependency installation and builds are included; context drift, in which the agent works from `main` at one commit while the target branch advances, making merge conflicts frequent.[S2] [S1] positions clones as common in CI and remote cloud workers where the workspace is a standalone disposable directory, and notes they are expensive for large monorepos or short tasks relative to worktrees; [S3] adds that on a single machine clones have been largely superseded by worktrees precisely because they duplicate the object store and re-track remotes independently, surviving mainly where worktrees cannot reach — different machines, each with its own hardware.

4.4 Containers and image snapshotting

Architecture. A container wraps the workspace — a bind-mounted worktree, or a clone created inside the container — with dependency, process, namespace, and resource isolation. [S2] details the snapshotting variant: a base image is built nightly from the repository's Dockerfile with dependencies pre-baked (`node_modules`, virtualenvs, compiled binaries); the agent runs in `runc`/`containerd`/gVisor with OverlayFS providing a thin copy-on-write read-write layer; progress is checkpointed with `docker commit <container> <image:step-5>`, and subsequent steps resume from the committed image, so a two-minute `npm run build` is instantly visible to the next step. Each container receives its own loopback interface, eliminating port conflicts.[S2] Warm starts under roughly two seconds are reported for this pattern, versus a minute-scale cold clone.[S2]

Documented implementations. [S3] describes Sculptor: one Docker container per agent built from the repository's devcontainer specification, with dependencies baked into a cached image layer so new agents start in seconds; container snapshots at turn boundaries for rollback; merge-conflict detection with resolution handed back to the agent; and a bidirectional sync mode that mirrors a container's state into the operator's local IDE for testing without the agent touching the host.[S3] Dagger's Container Use exposes the same pattern as an open-source MCP tool — a containerized sandbox plus a Git worktree per agent, with list/watch/log/diff inspection commands.[S3] OpenHands runs its agent loop against pluggable "runtimes" (a local Docker image, or remote runtimes such as Modal), and its documentation warns explicitly against running without the sandbox, since the agent then has full host filesystem access.[S3][11] [S2] documents

Factory's "persistent compute": long-running containers per developer or agent, with OverlayFS snapshots used to branch state — a mainline container is snapshotted, agents diverge from the snapshot, and merging happens at the Git level while filesystem state can be replayed on a fresh base.[S2] [S1] notes Anthropic's own guidance for unattended operation of its CLI: run it inside a devcontainer.[S1]

The shared-daemon problem. Both [S1] and [S3] document the central caveat of container-per-agent designs on one host: if each agent's container can reach the host Docker daemon — above all via a mounted socket — containers collide on names, networks, volumes, and port bindings, and an agent with daemon access can start a privileged container with host mounts and walk out of the sandbox entirely.[S1][S3] Documented countermeasures include one containerized database with a separate logical database per agent, database-branching services (a branch per workspace), and daemon-per-sandbox designs in which each sandbox receives a private, microVM-backed Docker daemon, which also solves docker-in-docker without privileged mode.[S3] [S1]'s hardening baseline for containerized agents: run as non-root with dropped capabilities; no Docker socket, no privileged mode, no host PID namespace; only the task workspace writable with a read-only root filesystem where practical; CPU, memory, process-count, disk, and time quotas; short-lived repository-scoped credentials; and default-deny or allowlisted egress.[S1] Empirical work supports the caveat: the SandboxEscapeBench evaluation found frontier models reliably exploit exactly these misconfigurations — exposed Docker sockets, privileged containers, dangerous capabilities — while correctly configured and fully patched runtimes resisted all escape attempts in the study.[4]

Documented trade-offs. Strengths: reproducible images (every agent starts from the same toolchain); isolated dependencies, environment variables, and network namespaces; CPU and memory limits; easier cleanup; snapshot-based stateful steps and rollback; a stronger boundary than raw host processes.[S1][S2] Limitations: first-build cost in minutes; image bloat as snapshot layers accumulate, requiring garbage collection and re-basing; disk and RAM multiplying per agent; orchestration complexity (registry, build pipeline, scheduler); the shared host kernel, which keeps kernel exploits in the threat model and motivates the runtimes of Section 4.6; and operational difficulty around stateful services — [S3] quotes practitioners on multi-workspace container setups colliding on compose project names, shared Postgres instances, host-wide port bindings, and background processes tied to session lifetimes.[S1][S2][S3]

4.5 Copy-on-write filesystem branching

Architecture. Rather than copying a workspace, copy-on-write (CoW) approaches give each agent a writable *delta* over a shared, read-only base. [S2] describes the OverlayFS/VFS variant used by cloud IDEs (Replit, Gitpod, StackBlitz): a single authoritative backend store holds the base, and each agent mounts an OverlayFS view with `lowerdir=readonly:/base/repo@commit`, `upperdir=rw:/agents/<id>/delta`, and a work directory; the kernel module (or `fuse-overlayfs`/`virtio-fs` in userspace) redirects writes to the upper layer.[27] Snapshotting is a metadata copy and effectively instantaneous; a background process syncs upper layers to object storage.[S2] The same principle appears at other levels of the stack: APFS `cp -c` clones duplicate a local directory tree (including `node_modules`) at near-zero cost for read-heavy workloads[S3]; Factory branches whole container filesystems from snapshots[S2]; and [S1] describes an emerging class of environment-level snapshot forking in which a warm development environment — files,

installed dependencies, processes, application state, and caches — is forked cheaply for several agents, with only modified pages or blocks copied.

VM-level forking. At the research frontier, [S3] documents ZeroBoot-style CoW virtual-machine forks: a golden VM snapshot is memory-mapped with `MAP_PRIVATE` and KVM state is forked, reportedly yielding sub-millisecond sandbox spawns (0.79 ms) with roughly 265 KB of marginal memory per fork — orders of magnitude beyond what container snapshotting achieves.[S3]

Documented trade-offs. Strengths: fork-speed workspace creation; a single shared base copy of dependency trees; native persistence across container restarts; whole-workspace rollback; support for speculative parallelism and GUI or browser agents.[S1][S2] Limitations: Linux-kernel dependency (awkward on macOS/Windows development machines without a VM); distributed-locking complexity on metadata servers when scaled horizontally; lower-layer staleness (existing overlays do not see base updates until remounted); difficult consistency semantics and complex selective merge for whole-environment forks; external side effects that cannot be rolled back; and limited maturity compared with Git and container workflows.[S1][S2]

4.6 Sandboxed runtimes, microVMs, and full VMs

Architecture. This family strengthens the boundary below the container. [S3] lays out the implementation spectrum. Conventional containers isolate via namespaces and cgroups on a shared host kernel — fast and cheap, but a kernel vulnerability is a cross-tenant escape. gVisor re-implements the Linux syscall surface in a userspace Go kernel so the workload never talks to the host kernel directly[3], eliminating the shared-kernel escape at the cost of some overhead and incomplete compatibility for low-level tools; Kata Containers similarly harden the boundary while retaining OCI and Kubernetes workflows.[S1][S3] Firecracker microVMs give each execution its own guest kernel via KVM with a deliberately minimal device model — roughly 100K lines of VMM code versus about two million for QEMU, six virtio devices, sub-125 ms boot, and about 5 MB memory overhead per VM.[2][S3] Full per-task VMs (Jules, Devin, Codex cloud, Copilot coding agent) sit at the end of the spectrum: a complete machine is provisioned, used, and destroyed.[S1]

The startup-latency question. Historically the decisive argument against VM-grade isolation was startup time; [S3] documents how the infrastructure tier collapsed it. Pre-warmed snapshot pools (boot ahead of demand, snapshot memory, restore on request) bring Firecracker starts to roughly 80–200 ms in E2B's case; userspace-kernel starts are reported around 100 ms; checkpoint/restore systems such as Fly.io Sprites resume persisted microVMs in roughly 300 ms while retaining 100 GB volumes. Figure 3 reproduces the vendor-reported figures gathered in [S3], which the source cautions are warm-path claims and can be 10–100× slower without pre-warming.[S3]

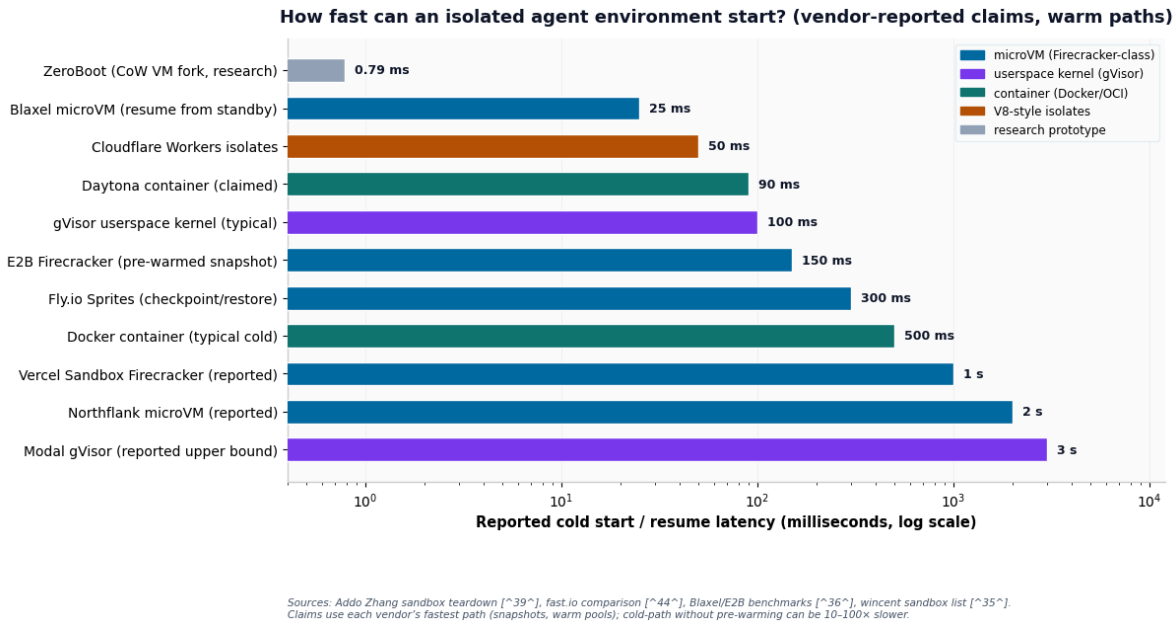

Figure 3. Reported cold-start or resume latencies by sandbox technology, as compiled in [S3] from vendor documentation and third-party comparisons (warm paths with pre-warmed snapshots; log scale). Figures are vendor-reported claims, not independent benchmarks.

Operational models. [S3] records that persistence models diverge sharply across providers: session-based sandboxes with pause/resume/snapshot (pausing billed or timed per GiB of RAM, session caps in the 1–24 hour range on managed tiers) versus stateful, long-lived workspaces; that secrets handling has converged on credential-proxy patterns that inject scoped, short-lived tokens at request time rather than baking long-lived keys into sandbox images; and that GPU support inside sandboxes is available only on some providers, which matters for agents running local models.[S3]

Security evidence. Two empirical anchors appear in the synthesized material. First, the Firecracker design literature documents the minimal-device-model, Rust-VMM approach that makes per-task microVMs economically viable at serverless scale.[2] Second, SandboxEscapeBench provides direct evidence about the container boundary this family is designed to replace: across 18 scenarios spanning orchestration, runtime, and kernel layers, frontier models exploited common misconfigurations (exposed Docker socket, privileged mode, writable host mounts) at or near 100% success, succeeded at meaningful rates on intermediate CVE-based scenarios, and solved none of the hardest kernel scenarios under the tested budgets; every successful escape used a previously disclosed vulnerability, and correctly configured, fully patched runtimes were not escaped.[4] [S1] positions gVisor/VM/microVM boundaries as the documented choice for multi-tenant platforms, sensitive credentials, hostile code, or regulated environments, while noting that no runtime boundary addresses Git or semantic conflicts.[S1]

4.7 OS-level process sandboxes

Architecture. Distinct from all workspace mechanisms, OS-level sandboxes constrain what a single agent *process* may touch, using primitives built into the operating system. [S3] documents Anthropic's open-source `sandbox-runtime` (`srt`): commands are wrapped with dynamically generated Seatbelt profiles via

`sandbox-exec` on macOS, and with bubblewrap[29] plus network namespaces on Linux. The policy model is dual and secure-by-default: filesystem writes are allow-only (denied everywhere unless explicitly granted), reads follow a deny-then-allow pattern, and network access is allow-only, routed through HTTP/SOCKS5 proxies on the host that enforce domain allowlists — on Linux the sandbox's network namespace is removed entirely, so the only path out is a bind-mounted Unix socket to the proxy.[S3][5] OpenAI's Codex CLI implements the same philosophy with different primitives — Landlock[28] and seccomp on Linux, Seatbelt on macOS, restricted tokens and ACLs on Windows — with network disabled by default and writes limited to the active workspace.[S3][7] Measured overhead is a fixed per-invocation cost on the order of 80–160 ms.[S3]

Documented usage. [S3] records that Anthropic stacks containment mechanisms by product surface — gVisor for the web product, Seatbelt/bubblewrap for the local CLI, and a full VM (Apple's Virtualization.framework on macOS, HCS on Windows) for its desktop agent — under the design principle that "if credentials never enter the sandbox, they can't be exfiltrated."[S3] Because it cages the worker rather than separating the work, this mechanism is documented as a *complement* to workspace isolation: an agent can be sandboxed inside its own worktree, gaining layer-B separation from the worktree and layer-C/E containment from the sandbox.[S3]

Documented trade-offs. Strengths: near-zero startup cost; no daemon or VM required; kernel-enforced rather than advisory; composable with any workspace strategy; reusable for arbitrary subprocesses and tool servers. Limitations: nothing for the parallel-work problem by itself (two sandboxed agents still collide in a shared directory); fiddly policy authoring; soft failure modes — the CLI warns and runs unsandboxed when prerequisites are missing unless configured to fail closed; and uneven platform coverage (Linux requires bubblewrap and socat; Windows support varies by tool).[S3]

4.8 Logical isolation at the orchestration layer

Architecture. [S2] draws the crucial distinction for frameworks such as LangGraph, Temporal, and AutoGen: they do not provide filesystem or process isolation at all — they provide *logical and state isolation*. Each agent run is a distinct thread or run ID; checkpointers persist state (messages, tool calls, serialized file diffs) to Postgres/Redis/S3 per thread; actor-model frameworks (Akka, Orleans, Temporal) serialize message processing through mailboxes. Actual code execution is delegated to a pluggable sandbox executor — E2B, Modal, Docker, Fly.io, or a local subprocess — so the orchestration layer is insulated from the choice of infrastructure primitive.[S2]

```
# LangGraph node pattern (from [S2])
def coder_agent(state):
    sandbox = pool.acquire()                # 1. request sandbox from pool
    # 2. sync git state:  sandbox.git.checkout(state['branch'])
    # 3. apply diffs from state
    # 4. run tools
    # 5. capture diff -> save to state
    # 6. release sandbox
    return {"diff": new_diff, "sandbox_id": sandbox.id}
```

Control planes. [S3] adds the terminal-multiplexer layer (Claude Squad, amux, Herdr, Gas Town's session layer), which solves process persistence and observability — agents keep running when the operator detaches, and their working/blocked/done state is visible — but provides no isolation of its own and is explicitly documented as composing with any of the primitives above: panes over worktrees locally, remote sandboxed agents attached over SSH, all in one view. Herdr additionally exposes a JSON-over-socket API that lets agents themselves create panes and spawn helpers, and a remote mode that makes a distant server feel local.[S3][24]

Documented trade-offs. Strengths: durability, retries, visibility, human-in-the-loop gates, and executor portability — the agent logic is unchanged when swapping local Docker for a cloud sandbox provider.[S2] Limitations: the framework supplies no containment by itself — safety is entirely a property of the executor chosen; and coordination correctness (claims, leases, state transitions) must still be engineered. [S1] generalizes the point into a design rule documented in its reference architecture: deterministic software, not agent judgment, should be the authority for task claims, leases, state transitions, workspace creation, permission policy, retries, cleanup, and merge gates.[S1]

5 Cross-Cutting Implementation Challenges

Independently of the mechanism chosen, the sources document a recurring set of operational problems. This section consolidates them; Table 4 summarizes.

5.1 Dependency drift

A base image pins one dependency version; an agent upgrades it inside its own snapshot; the next agent starts from the base and expects the old version. [S2] documents the mitigations in use: immutable bases with lockfile-driven installs (`uv sync`, `npm ci`) at the cost of startup time; per-worktree editable installs; and deterministic, hash-addressed environment systems (Nix, devboxes, Flox — used by Replit and Determinate Systems among others) that create isolated environments instantly.[31][S2]

5.2 Untracked-file propagation

Worktree and clean-clone approaches materialize tracked files only, so `.env` files, credentials, and build inputs silently go missing. Documented remedies: per-workspace setup scripts that install dependencies and copy environment files; explicit "files to copy" manifests (Conductor's `.worktreeinclude`); CoW directory clones on APFS; and shared symlinks where dependency sets are identical.[S3]

5.3 Long-running processes and shared services

Agents start watchers and databases in the background; in worktree setups these collide on ports, and in container setups they die with the container's pause or snapshot. [S2] documents the sidecar pattern: the orchestrator manages service containers (database, cache, mock server) shared via service-mesh DNS, and agents connect to shared services while only the code filesystem is isolated.[S2] [S3] documents the per-workspace alternative — database branching (a Neon/Supabase branch per workspace[30]) or a dedicated

compose project per task[13] — and [S1] prescribes unique database names, volume names, cache prefixes, queue namespaces, and dynamic ports as the general external-state isolation discipline.[S1][S3]

5.4 The shared daemon and control-plane access

Covered in Section 4.4: a mounted Docker socket converts container isolation into host access, and is the first item in every hardening baseline; it is also among the most reliably exploited misconfigurations in the empirical escape literature.[4]

5.5 Secrets and identity isolation

Agents need credentials — cloud keys, API tokens, database passwords. [S2] contrasts the bad pattern (static keys injected as environment variables, visible via `docker inspect`) with the documented good patterns: secrets managers mounted via CSI drivers into specific container paths; OIDC/workload identity issuing short-lived tokens per pod or service account (AWS IRSA, GCP Workload Identity Federation, SPIFFE) so no static keys exist; and, at the sandbox-infrastructure tier, credential proxies injecting scoped tokens at request time.[S2][S3] The design principle recorded across sources is that credentials should never enter the sandbox at all where avoidable.[S3]

5.6 Per-workspace context and indexing

Isolation multiplies workspaces, and each agent needs codebase context; re-indexing `node_modules` fifty times is prohibitively expensive. [S2] documents the shared read-only index pattern: build the code graph (LSIF/SCIP) once on the base image, mount it read-only into all agents, and have agents index only their own deltas.[S2]

Table 4. Cross-cutting implementation challenges and documented mitigations

Challenge	Failure mode	Documented mitigations
Dependency drift	Agents diverge from base image; snapshots expect different versions	Immutable base + lockfile installs; per-workspace editable installs; Nix/Flox deterministic environments[S2]
Untracked files	`.env`, credentials, build inputs missing in fresh workspaces	Setup scripts; files-to-copy manifests; APFS CoW clones; shared symlinks[S3]
Long-running services	Port conflicts in worktrees; processes die with container snapshots	Sidecar/shared services via DNS; per-workspace DB branches or compose projects; unique namespaces and dynamic ports[S1][S2]
Daemon/control-plane access	Mounted Docker socket = host access; container name/port collisions	No socket mounts; daemon-per-sandbox; hardening baselines[S1][4]
Secrets and identity	Static keys in env vars; shared accounts; exfiltration	CSI-mounted secrets; OIDC/workload identity; credential proxies; no-credentials-inside design[S2][S3]

Context indexing	Re-indexing cost multiplied per workspace	Shared read-only base index + per-agent delta indexing[S2]

6 Integration and Coordination

All three sources stress the same boundary: isolation mechanisms govern what happens *while* agents work; they say nothing about whether the work comes back together correctly.[S1][S3] This section consolidates what the sources document about the integration layer.

6.1 What isolation does not solve

Semantic conflicts. Two agents may edit different files while making incompatible assumptions — [S1]'s example: one migrates an identifier from integer to UUID while another builds analytics code that still expects integers. Git reports no textual conflict, and the combined system fails; only repository-wide integration tests and architectural review catch it.[S1] [S3] generalizes: worktrees prevent file collisions, not logic collisions, and the documented heuristic is "different files → parallel; same files → sequential or split first."[S3]

Database and schema migrations. [S1] calls parallel migration work unusually risky: agents may reuse migration sequence numbers, rename the same field differently, produce incompatible forward paths, or update readers and writers in the wrong order; such work generally requires explicit dependency ordering and backward-compatible staging.[S1]

Global configuration hotspots. Central dependency manifests, lockfiles, route registries, generated clients, and dependency-injection configuration merge cleanly as text yet diverge in ordering and generated-state assumptions; [S1] documents designated ownership or short-lived leases for these resources.[S1]

The moving base branch. A task can succeed against an old commit while other changes land; documented practice is to update the branch against the latest target, rebuild, and retest before merge, and to use a merge queue that tests queued changes against current target state and serializes landing.[S1][33]

External side effects. Snapshots and disposable VMs cannot undo emails, cloud mutations, package publication, production API calls, or writes to shared SaaS accounts; documented countermeasures are scoped credentials, fake services, and explicit egress policy.[S1]

The merge wall. At fleet scale the integration cost itself becomes the constraint: [S3] documents that with twenty parallel workers producing twenty branches, every merge shifts the baseline under the remaining pull requests, and that practitioner reports place reliable day-to-day operation at roughly three to five concurrent agents, with failure modes in 30–40% of attempts when pushing ten agents on one repository in early multi-agent features — human review bandwidth, not git, becomes the binding constraint.[S3]

6.2 Documented coordination strategies

Table 5 merges the coordination taxonomies from [S1] and [S3]. Three merge-management patterns documented in the 2026 tool landscape deserve explicit note. (i) *Dedicated merge agents*: Gas Town employs

a "Refinery" role that polls a review queue, merges what it can, and routes conflicts back to the original worker through a git-backed issue ledger whose hash-based IDs are themselves designed to avoid merge conflicts.[S3] (ii) *CI ratchets*: Multiclaude merges automatically when CI passes and reassigns failing pull requests to their authors — a "Brownian ratchet" in which forward progress is continuous but every landing is gated by the test suite.[S3] (iii) *Pre-merge conflict prediction*: tooling such as Clash runs `git merge-tree` across in-flight worktrees to detect collisions before they reach integration.[S3] Complementing these, [S3] documents the reviewer-gate pattern — roughly one read-only reviewer agent per three to four builders, auto-triggered on task completion, so the human lead sees only pre-reviewed code — and [S2] documents stacked/dependent branches, nightly auto-rebase automation that "wakes" an agent to resolve conflicts, and monorepo tooling (nx, turborepo, Bazel) that gives agents isolated project subgraphs with defined boundaries.[S2]

Table 5. Coordination strategies documented across the sources (merged from [S1] and [S3])

Strategy	How it works	Strength	Main risk
Independent branches	Each task starts from the same base and merges later	Maximum parallelism; simple dispatch	High conflict and duplication risk for long or coupled tasks
Dependency graph	Tasks declare prerequisites; only unblocked work runs	Correct ordering for migrations and staged refactors	Decomposition can be wrong or too serial
Ownership partitions	Agents receive directory or component boundaries	Reduces overlap in modular codebases	Cross-cutting work and shared interfaces remain difficult
Resource leases	Lock a file, schema, migration stream, or release manifest for a period (expiring, with heartbeats)	Prevents direct concurrent mutation of hotspots	Deadlocks, stale locks, reduced concurrency
Coordinator–worker	Parent agent decomposes, monitors, reviews, and combines worker output	Dynamic scope; specialist workers	Coordinator bottleneck, token cost, non-determinism
Speculative parallelism	Several agents solve the same task; a judge selects or synthesizes (e.g., best-of-n worktree races)	Improves difficult-task reliability; model comparison	Multiplies compute; complicates evaluation
Merge queue / ratchet	Serialize landing; test each change against the current target; auto-merge on green CI	Target branch stays green; ordering explicit	Queue latency; CI cost per landing

[S1] adds two usage notes: locks are most effective for narrow, high-conflict resources (migration sequences, lockfiles, release manifests, generated schemas) and should be expiring leases with heartbeats rather than permanent locks; and ownership partitions work best as a scheduling and review signal rather than as the only safety mechanism, with boundary-expansion requests recorded and checked for overlap with active work.[S1]

7 Product and Framework Landscape

Table 6 merges the landscape surveys of [S1] and [S3], focusing on the advertised workspace and runtime boundaries rather than model quality. The picture that emerges is a spectrum from local Git-aware tools to fully managed per-task cloud environments, with several products spanning multiple tiers (Cursor's local worktrees plus managed cloud; Codex's local sandbox, worktree mode, and per-task cloud containers). Cloud development environment platforms exhibit the same control-plane/workspace split: Coder's documentation describes workspaces as standard compute infrastructure with no agent software installed — the agent loop runs in the control plane, and a workspace is provisioned only when the agent needs to take action.[32]

Table 6. Product and framework landscape: documented isolation boundaries (merged from [S1] and [S3]; capabilities as documented at synthesis time and subject to change)

System	Access style	Source isolation	Runtime isolation	Integration
Aider[10]	Direct API / BYOK	Current checkout unless user supplies branch, worktree, or clone	Local host by default	Automatic Git commits, undo, user-managed merge
Claude Code[6]	Subscription and API workflows	Worktrees for parallel sessions or isolated subagents (`--worktree`)	Local environment; optional OS sandbox (`srt`)	Branch/worktree review and integration
Codex (CLI/app/cloud)[7]	Subscription-managed plus API	Local checkout, worktree mode, or isolated cloud checkout	Local OS sandbox (Landlock/seccomp/Seatbelt) or managed cloud environment per task	Diff, commit, push, PR, follow-up task
Cursor[26]	Subscription with model choice	Local Git worktrees (`/worktree`); managed cloud-agent modes; background agents	Local, cloud sandbox, or remote SSH per session	Review and integrate agent changes; apply-worktree flow
GitHub Copilot coding agent[8]	GitHub subscription	Task branch in its own environment	Ephemeral managed environment per task	Pull request, checks, review
Google Jules[9]	Managed service	Repository clone and source branch per task	VM per task; environment snapshots; VM destroyed after task	Push branch; PR review in GitHub; CI-failure fix loop
Devin[12]	Managed service	Session-specific workspace	Isolated VM per worker; environment snapshotting	Coordinator monitors and resolves conflicts
OpenHands[11]	Model-agnostic / self-hostable	Deployment-defined workspace	Docker runtime, remote runtimes (e.g., Modal), or unsafe host process (warned against)	Deployment-defined branch and PR flow

Sculptor[S3]	Subscription/API passthrough	Clone inside per-agent container	Docker container per agent from cached devcontainer image; turn snapshots	Merge-conflict detection; IDE sync for local testing
Container Use (Dagger)[19]	Open-source tool (MCP)	Worktree per agent	Containerized sandbox per agent	Branch diff/log inspection; user-managed merge
Conductor[22]	Passthrough of existing CLI login (API or subscription)	Worktree per workspace; files-to-copy for gitignored files	Local host	Branch review in app
Superset[21]	Agent-agnostic desktop (any CLI)	Worktree per task; setup/teardown scripts (deps, env, DB branches)	Local host; cloud workspaces planned	Branch review; diff viewer
Claude Squad[20]	Terminal (tmux)	Worktree per agent	Local host; tmux panes as control plane	Branch review
Vibe Kanban[S3]	Agent-agnostic	Worktree-backed workspaces	Local host	Kanban review flow
Gas Town[23]	Orchestrator over CLI agents	Worktree "hooks" per worker; persistent state	Local; tmux session per worker	Refinery merge agent; git-backed issue ledger (Beads)
Herdr[24]	Multiplexer (control plane only)	None itself; composes with worktrees/remote sandboxes	Local or remote via SSH	Operator-managed
JetBrains Air[S3]	Desktop	Worktree or container per task	Local Docker or host	Task review in IDE
AutoGen[14]	Model-agnostic framework	Application-defined	Optional Docker code executor	Application-defined
LangGraph / Temporal[15][16]	Model-agnostic frameworks	Application-defined	Pluggable sandbox executors	Durable state, retries, human gates
Symphony[S1]	Agent orchestrator (spec)	Isolated per-issue workspace	Runner implementation-defined	Issue-driven workflow; CI evidence; landing policy
E2B / Modal / Daytona / Northflank / Vercel / Fly[17][18]	Sandbox infrastructure (SDK)	Template-defined	Firecracker microVM, gVisor, or containers per session	None (infrastructure primitive)

7.1 Access models: direct API versus subscription

All three sources address the access dimension, and [S1] states the central analytical point: the payment model does not determine the isolation model — subscription products commonly bundle model access with workspace provisioning, execution infrastructure, logs, branch creation, and review workflows, while direct-API and self-hosted approaches expose more implementation control and transfer provisioning, security, observability, cleanup, and integration policy to the operator; the decisive question remains whether every concurrent task receives a separate writable workspace, runtime boundary, external-service namespace, and integration path.[S1]

[S3] documents the 2026 policy landscape that constrains how credentials may be combined with harnesses: in January 2026, Anthropic blocked third-party harnesses from spending Claude subscription quota (third-party tools reimplementing the harness must use metered API access, while surfaces connected through the Agent Client Protocol count as first-party); OpenCode documents support for ChatGPT Plus, GitHub Copilot, and GitLab Duo subscription logins; and wrappers that drive first-party CLIs inherit whatever those CLIs are authenticated with — Conductor's documentation states it uses an API key or a Pro/Max subscription depending on the operator's existing Claude Code login.[S3][25][22] Two fleet-level consequences are recorded: subscription rate limits are per-account, so parallel agents share one token budget; and local gateway proxies are used to centralize authentication (subscription passthrough or key routing), apply per-agent rate limits, and provide a single observability point.[S3]

8 Documented Usage Patterns

The sources each map recurring scenarios to mechanism stacks. These mappings are reproduced here as documented patterns, with attribution; they represent the sources' analyses of where each stack has been applied, not this paper's endorsement. [S2] makes its mapping criteria explicit as three decision axes: the **trust boundary** (trusted in-house agents versus untrusted user code — the source calls microVM/gVisor isolation "non-negotiable" for the untrusted case), the **latency budget** (sub-two-second interactive starts versus minute-scale batch starts), and **statefulness** (whether agents need long-running companion services).[S2]

Table 7. Documented usage patterns: scenario-to-stack mappings from the sources

Scenario	Documented stack	Documented rationale	Source
Batch / asynchronous fleets (unattended bug-fixing, migrations)	Pre-baked container images with snapshotting on an orchestrated fleet; worktrees inside containers; draft PRs with a merge bot and conflict-resolution sub-tasks; or managed async agents (Jules, Codex cloud) with per-task VMs	Warm starts; bin-packing efficiency; hard isolation; PR-per-task audit trail; no local resource use	[S2][S3]

Interactive local assistance (a few agents alongside the operator)	Git worktrees plus terminal-multiplexer panes and IDE multi-root workspaces; desktop worktree orchestrators, optionally with an OS process sandbox	Zero latency; local compute; the operator resolves conflicts instantly	[S1] [S2] [S3]
Multi-tenant agent SaaS (users bring repositories; agents act on them)	MicroVMs (Firecracker-class) or gVisor/Kata per task or tenant; OverlayFS storage; OIDC/workload identity; credential proxies; strict egress	Hard multi-tenancy: one tenant's agent cannot reach another's filesystem, network, or secrets; kernel exploits contained	[S1] [S2] [S3]
Multi-step pipelines (plan → code → test → deploy)	Temporal/LangGraph-class durable orchestration with a pluggable sandbox executor	Durability, retries, visibility, human gates; isolation delegated to the executor tier	[S2]
Large monorepo at high concurrency	Shared Git object storage, prewarmed mirrors, worktrees or CoW materialization; ownership partitions and dependency graphs; serialized hotspots; per-task build-cache namespaces	Avoids repeated clone/setup cost; boundaries match project subgraphs (nx/turborepo/Bazel)	[S1] [S2]
Database or protocol migration	Backward-compatible schema/protocol staging; readers updated before writers; backfill and verification; compatibility removed only after dependents migrate	Parallel migration work documented as unusually risky; ordering is explicit	[S1]

[S1] condenses its analysis into four dominant designs observed across engineering organizations: (i) a shared directory with logical roles, cheap and usable only with a strictly enforced single writer; (ii) a Git worktree per agent, the lightweight source-isolation method documented for trusted local development; (iii) a worktree or clone plus a container per agent, the strongest general-purpose self-hosted architecture in its survey; and (iv) an ephemeral VM or microVM per agent, the boundary documented for multi-tenancy, hostile code, sensitive credentials, and managed cloud services.[S1] [S3] adds the access-model constraint as a practical axis: operators on subscription plans are documented as favoring orchestrators that wrap first-party CLIs (which inherit subscription authentication), while API-key operators have unrestricted harness choice.[S3]

9 Convergence Trends

The sandbox as a commodity API. [S2] documents convergence across sandbox providers (E2B, Modal, Daytona, Browserbase, CodeSandbox) on a common interface shape — start from a template, run code, read and write files, snapshot for forking, expose ports, close — with orchestrators becoming sandbox-agnostic so the executor backend can be swapped without changing agent logic.[S2]

```
// The converging sandbox interface shape (from [S2])
interface Sandbox {
  start(template: "python" | "node" | "dockerfile"): Promise<Handle>;
  runCode(handle, code: string): Promise<Result>;
  writeFiles(handle, files: Map<string, string>);
  readFiles(handle, paths: string[]): Map<string, string>;
  snapshot(handle): Promise<ImageRef>;   // for forking
  exposePort(handle, port: number): string;
  close(handle);
}
```

Local and cloud isolation are merging. [S3] documents tools born local adding remote backends (planned cloud workspaces in desktop orchestrators; multiplexer remote-attach modes; local coordinators provisioning ephemeral cloud development environments per delegated task), while cloud-born tools add interactive synchronization (container-to-IDE pairing, one UI over local/cloud/SSH/worktree session types). The documented end state is a single control plane over a continuum of isolation strengths chosen per task.[S3]

Isolation is becoming default-on. First-party CLIs now ship both worktrees and OS-level sandboxes built in, and the underlying primitives are open-source commodities (sandbox-runtime, Firecracker, gVisor) rather than differentiators.[S3] [S3] also records the competitive implication: differentiation has moved up-stack to coordination and merge machinery — git-backed state ledgers, dedicated merge agents, CI ratchets, conflict prediction, and reviewer gates (Section 6).

Landscape volatility. As a caveat for any landscape table, [S3] documents rapid churn in this category during 2026: one sandbox provider took its production codebase closed-source, and one orchestrator's maintaining company shut down with the project continuing under community maintenance. Product capabilities in Table 6 are a snapshot, and [S1]'s guidance is to verify contractually guaranteed isolation properties rather than implementation details that may change.[S1][S3]

10 Summary

The three synthesized sources, produced independently, arrive at the same structural account. Isolation for concurrent coding agents is a stack, not a feature: context, source tree, process/runtime, external state, credentials/network, and integration are separable layers, and mechanisms address different layers. Nine mechanism families cover the design space — shared-checkout conventions, branch-only separation, Git worktrees, full clones, containers with image snapshotting, copy-on-write filesystem branching, sandboxed runtimes and microVMs, OS-level process sandboxes, and logical orchestration-layer isolation — and deployed systems stack several of them. Worktrees dominate local source isolation; containers add dependency and namespace isolation with a well-documented daemon-access caveat; microVM and userspace-kernel sandboxes provide the kernel-level boundary documented for untrusted and multi-tenant execution, with pre-warmed snapshots having collapsed their historical startup penalty; OS-level process sandboxes cage individual agents orthogonally to any workspace strategy; and orchestration frameworks contribute state isolation while delegating containment to pluggable executors. No mechanism in any family resolves the integration problem — semantic conflicts, migration hazards, moving base branches, and

irreversible external side effects persist under perfect runtime isolation — which is why the documented systems pair every isolation primitive with coordination machinery: dependency graphs, ownership partitions, expiring leases, merge queues, CI ratchets, dedicated merge agents, and reviewer gates. The access model (direct API versus subscription) constrains credential use and operational responsibility but does not, in itself, determine isolation properties. As the sources' shared conclusion puts it: the engineering system around the agents — not the agents alone — determines whether parallel work is safe, reproducible, and mergeable.[S1]

References

Source documents synthesized

[S1] *Multi-Agent Coding Isolation: Architectures, implementations, and trade-offs for concurrent AI software agents* (research report). Research snapshot July 16, 2026.

[S2] *Isolation strategies for multi-agent AI coding systems on a shared codebase* (research compilation, `isolation-research.md`). 2026.

[S3] *Isolation Approaches for Parallel AI Coding Agents — A Deep Research Report*. July 17, 2026.

Cited works

1. Git Project. (n.d.). *git-worktree — Manage multiple working trees* (Git documentation). https://git-scm.com/docs/git-worktree
2. Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., & Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)* (pp. 419–434). USENIX Association.
3. gVisor Authors. (n.d.). *gVisor documentation*. https://gvisor.dev/docs/
4. Marchand, R., et al. (2026). Quantifying frontier LLM capabilities for container sandbox escape (SandboxEscapeBench). *arXiv:2603.02277*. University of Oxford / UK AI Security Institute. https://arxiv.org/abs/2603.02277
5. Anthropic. (2026). *sandbox-runtime (srt)* [Computer software]. GitHub. https://github.com/anthropics/sandbox-runtime
6. Anthropic. (2026). *Claude Code documentation* (worktrees, subagents, sandboxing). https://docs.anthropic.com/en/docs/claude-code
7. OpenAI. (2026). *OpenAI Codex documentation* (CLI, cloud, and environment modes). https://developers.openai.com/codex
8. GitHub. (2026). *About Copilot coding agent* (GitHub Docs). https://docs.github.com/en/copilot
9. Google. (2026). *Jules documentation*. https://jules.google/docs
10. Aider. (2026). *Aider documentation* (Git integration; architect mode). https://aider.chat/docs/
11. OpenHands Contributors. (2026). *OpenHands documentation* (runtimes and sandboxing). https://docs.all-hands.dev/
12. Cognition AI. (2026). *Devin documentation*. https://docs.devin.ai/
13. Docker Inc. (n.d.). *Docker Compose documentation* (projects and networking). https://docs.docker.com/compose/
14. Microsoft. (2026). *AutoGen documentation* (code executors). https://microsoft.github.io/autogen/

15. LangChain. (2026). *LangGraph documentation* (persistence and checkpointers). https://langchain-ai.github.io/langgraph/
16. Temporal Technologies. (2026). *Temporal documentation*. https://docs.temporal.io/
17. E2B. (2026). *E2B documentation* (Firecracker sandboxes). https://e2b.dev/docs
18. Modal Labs. (2026). *Modal Sandboxes documentation*. https://modal.com/docs
19. Dagger. (2026). *Container Use* [Computer software]. GitHub. https://github.com/dagger/container-use
20. smtg-ai. (2026). *Claude Squad* [Computer software]. GitHub. https://github.com/smtg-ai/claude-squad
21. Superset. (2026). *Superset* [Computer software]. GitHub. https://github.com/superset-sh/superset
22. Conductor. (2026). *Conductor documentation*. https://conductor.build/docs
23. Yegge, S. (2026). *Gas Town* [Computer software]. GitHub. https://github.com/steveyegge/gastown
24. Zechner, M. (2026). *herdr* [Computer software]. GitHub. https://github.com/badlogic/herdr
25. SST. (2026). *OpenCode documentation* (providers and authentication). https://opencode.ai/docs/
26. Anysphere. (2026). *Cursor documentation* (worktrees; background agents). https://cursor.com/docs
27. Kernel.org. (n.d.). *OverlayFS filesystem documentation*. https://docs.kernel.org/filesystems/overlayfs.html
28. Kernel.org. (n.d.). *Landlock: unprivileged access-control*. https://docs.kernel.org/userspace-api/landlock.html
29. Containers Project. (n.d.). *bubblewrap* [Computer software]. GitHub. https://github.com/containers/bubblewrap
30. Neon. (2026). *Neon branching documentation*. https://neon.tech/docs/
31. NixOS Foundation. (n.d.). *Nix & NixOS documentation*. https://nixos.org/learn/
32. Coder. (2026). *Coder Agents documentation*. https://coder.com/docs/ai-coder/agents
33. GitHub. (n.d.). *Managing a merge queue* (GitHub Docs). https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository

Note on sourcing: product capabilities, access policies, and startup-latency figures are as documented by vendors and by the three source documents at synthesis time (July 2026) and are subject to change; latency figures in Figure 3 are vendor-reported warm-path claims rather than independent benchmarks. Two works listed in [S1]'s further-reading section (a Git-scaling systems paper and a workspace-forking study) could not be independently verified during synthesis and are therefore not cited here.